

HMT N32 (Addendum): Especificación upstream de U_{12} y canonicidad de gauge

(certificado reproducible)

January 19, 2026

Objetivo

Este addendum cierra el hueco que un referee suele señalar cuando uno demuestra que “*Go-lay/Steiner está bien construido*” pero no deja cristalino el *punte* con el motor:

1. **Especificación upstream (sin outputs):** cómo se genera U_{12} desde APP/TPK/MOV3/ κ –27 con flip/twist/gauge.
2. **Variantes razonables $\mathcal{H}$:** qué opciones “naturales” existen (p.ej. el *twist* en la capa TIMES).
3. **Filtro interno $\mathcal{C}$:** restricciones internas que eliminan diales (no α).
4. **Resultado:** cuántas variantes sobreviven (canonicidad dura vs. parámetro real/gauge).

El paquete asociado incluye un script determinista y un certificado por sha256 de todos los artefactos generados.

1 Definición upstream: el extractor E que produce U_{12}

1.1 APP como toro 9×9 con dos capas

Trabajamos sobre un toro $\mathbb{Z}_9 \times \mathbb{Z}_9$ (coordenadas $(i, j) \bmod 9$).

Raíz digital mod 9.

$$\text{droot}_9(n) = \begin{cases} 9, & n \equiv 0 \pmod{9}, \\ n \bmod 9, & \text{si no.} \end{cases}$$

Capa PLUS y capa TIMES.

$$\text{PLUS}_9[i, j] = \text{droot}_9((i+1) + (j+1)), \quad \text{TIMES}_9[i, j] = \text{droot}_9((i+1)(j+1)).$$

(Así se codifica la “APP” en 9×9 sin parámetros.)

1.2 MOV3 (partición triádica de cada bloque de 9 ticks)

Cada evento tiene 9 ticks. Definimos un selector $\delta(t) \in \{+1, -1, 0\}$ para el tick global t :

$$\delta(t) = \begin{cases} +1, & t \bmod 9 \in \{1, 4, 7\} \quad (\text{PLUS}), \\ -1, & t \bmod 9 \in \{2, 5, 8\} \quad (\text{TIMES}), \\ 0, & t \bmod 9 \in \{3, 6, 0\} \quad (\text{torsion}). \end{cases}$$

1.3 κ -27 / flip: simetría a 27 ticks

El flip actúa cada 27 ticks: en el tick global $t \in \{27, 54, 81, 108\}$ se invierte el heading (signo del vector-dirección) de ambos caminantes.

1.4 Gauge: semillas y headings

La gauge de entrada son dos semillas y dos headings:

$$(i_P, j_P, h_P), (i_T, j_T, h_T), \quad i_*, j_* \in \{0, \dots, 8\}, h_* \in \{N, E, S, O\}.$$

Hay $9^4 \cdot 4^2 = 104\,976$ semillas.

1.5 Twist (“neighbor 3-adic”): log discreto en $(\mathbb{Z}/9\mathbb{Z})^\times$

La capa TIMES no se lee como “dígito crudo” sino a través de un *twist* LOG_9 que (i) mata raíces (múltiplos de 3) y (ii) linealiza la parte unitaria como log discreto.

Definición. Sea el grupo de unidades $(\mathbb{Z}/9\mathbb{Z})^\times = \{1, 2, 4, 5, 7, 8\}$ (orden 6). Elegimos generador canónico $g = 2$ (menor generador positivo). Definimos

$$\text{LOG}_9(x) = \begin{cases} 0, & x \in \{3, 6, 9\}, \\ \ell, & x \equiv g^\ell \pmod{9} \text{ con } \ell \in \{0, 1, 2, 3, 4, 5\}. \end{cases}$$

Con $g = 2$: $1 \mapsto 0, 2 \mapsto 1, 4 \mapsto 2, 8 \mapsto 3, 7 \mapsto 4, 5 \mapsto 5$.

Comentario. Este *twist* es interno (no usa α) y es exactamente el lugar donde un referee pregunta si hay “magia” o dial escondido. Por eso en la sección ?? enumeramos variantes y filtramos.

1.6 Formación de triadas base-1000

Para cada evento (9 ticks), acumulamos:

- S : suma de lecturas en capa PLUS cuando $\delta = +1$.
- E : suma de lecturas *twisteadas* $\text{LOG}_9(\text{TIMES}_9)$ cuando $\delta = -1$.
- T : contador de ticks con $\delta = 0$ (siempre $T = 3$ por evento).

Luego se emite una triada (tres dígitos decimales)

$$(d_1, d_2, d_3) = (S \bmod 10, (S+E) \bmod 10, (3S+5E+7T) \bmod 10), \quad u = 100d_1 + 10d_2 + d_3 \in \{0, \dots, 999\}.$$

U_6 y U_{12} . La implementación upstream genera $U_6 \in \{0, \dots, 999\}^6$ en 54 ticks (6 eventos). El modelo completo toma $U_{12} = U_6 \| U_6$ (doble hoja por el flip a 54/108).

2 Variantes $\mathcal{H}$ y filtro interno $\mathcal{C}$

2.1 $\mathcal{H}$: variantes “razonables” (sin tocar outputs)

En la práctica el único punto con ambigüedad razonable es el *twist* TIMES. Consideramos:

1. $\mathcal{H}_{\log 2}$: log discreto en unidades con generador $g = 2$ (canónico por “menor generador”).

2. $\mathcal{H}_{\log 5}$: log discreto con generador alternativo $g = 5$ (equivalente a inversión en el grupo de unidades).
3. $\mathcal{H}_{\text{val}3}$: valuación v_3 (cuenta factores de 3).
4. $\mathcal{H}_{\text{id}}$: identidad (usar TIMES crudo como incremento).

2.2 $\mathcal{C}$: restricciones internas

El filtro interno que un referee acepta sin apelar a α :

1. $\mathcal{C}_{\text{root}}$: **mata raíces**: $\text{LOG}_9(3) = \text{LOG}_9(6) = \text{LOG}_9(9) = 0$.
2. $\mathcal{C}_{\text{hom}}$: **homomorfismo de unidades**: para $a, b \in \{1, 2, 4, 5, 7, 8\}$,

$$\text{LOG}_9(ab) \equiv \text{LOG}_9(a) + \text{LOG}_9(b) \pmod{6}.$$
3. $\mathcal{C}_{\text{min}}$: **fijar generador canónico**: elegir el menor generador positivo de $(\mathbb{Z}/9\mathbb{Z})^\times$, que es $g = 2$.
4. $\mathcal{C}_{\text{catalog}}$: **catálogo finito rico**: el escaneo exhaustivo sobre 104 976 semillas debe dar exactamente 468 U_6 distintos (invariante interno reportable y reproducible).

2.3 Resultado del barrido (exhaustivo sobre semillas; variantes finitas)

El script `hmt_n32_upstream_u12_gauge_scan.py` ejecuta:

- un escaneo exhaustivo sobre las 104 976 semillas,
- para cada variante $\mathcal{H}_k$, calcula el número de U_6 distintos y estadísticas de fibras,
- emite un JSON con hashes `sha256` de los CSV producidos.

Variante	$ \{U_6\} $	Pasa $\mathcal{C}_{\text{root}} + \mathcal{C}_{\text{hom}}$	Comentario
$\mathcal{H}_{\log 2}$ (canónico)	468	sí	candidato canónico
$\mathcal{H}_{\log 5}$ (alternativo)	468	sí	gauge residual (inversión)
$\mathcal{H}_{\text{val}3}$	144	no	no mata raíces
$\mathcal{H}_{\text{id}}$	684	no	no es log/hom

Conclusión. Bajo $\mathcal{C}_{\text{root}} + \mathcal{C}_{\text{hom}}$ sobreviven dos twists (dos generadores del grupo cíclico de orden 6). El axioma interno $\mathcal{C}_{\text{min}}$ fija $g = 2$ y deja **un** módulo gauge canónico.

3 Artefactos y certificado

El certificado JSON `HMT_N32_UPSTREAM_U12_CERTIFICATE.json` incluye hashes `sha256` de:

- `n32_u6_seed_table.csv` (104 976 filas: semilla $\mapsto$ uid),
- `n32_uid_representative_seed.csv` (468 uids con semilla representativa),
- `n32_uid_u6_catalog.csv` (uid $\mapsto U_6$),
- `n32_variant_scan.json` (tabla de variantes $\mathcal{H}$ y checks $\mathcal{C}$).

Reproducibilidad. En una máquina limpia:

```
python hmt_n32_upstream_u12_gauge_scan.py
# produce CSV + JSON + hashes
```